1. Title Page

Lab Name: Lab 07: Internet Protocol (IPv4 and IPv6) Foundations **Date:** YYYY-MM-DD **Name:** [Your Name] **Lab Partners:** [Partners' Names, if any] **Software/Hardware Used:** Cisco Modeling Labs (CML) - Personal / Free Edition

---

2. Background

The Network Layer is responsible for host-to-host communication, routing, and addressing. The primary protocols operating at this layer are:

1. **Internet Protocol version 4 (IPv4)**: Operates with 32-bit addresses. The header is variable in size (typically 20 bytes) and includes fields to handle packet fragmentation (Identification, Flags, Fragment Offset) when a packet exceeds the link's Maximum Transmission Unit (MTU).
2. **Internet Protocol version 6 (IPv6)**: Developed to replace IPv4, utilizing 128-bit addresses. It simplifies routing and packet processing by using a fixed 40-byte header, removing the header checksum, and shifting optional features (like fragmentation) to extension headers. Crucially, IPv6 routers do not fragment packets in transit; instead, IPv6 hosts perform Path MTU Discovery (PMTUD).

In this lab, you will deploy a dual-stack routed topology in CML, generate standard and fragmented IPv4 traffic, generate IPv6 traffic, and analyze the resulting headers and fragments in Wireshark.

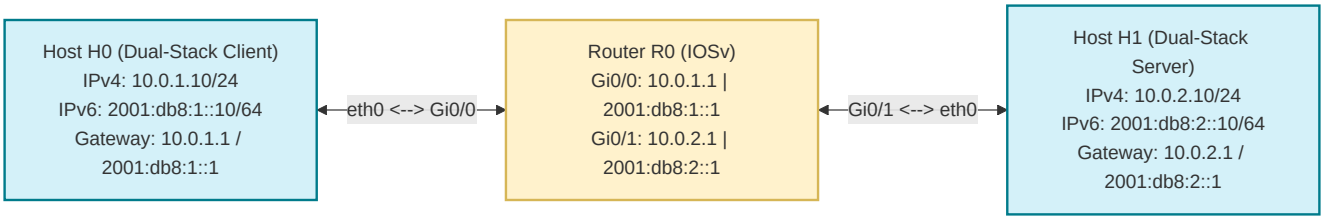
---

3. Objective / Purpose

To configure dual-stack IPv4/IPv6 interfaces on hosts and routers, analyze the fields of IPv4 and IPv6 headers, observe the mechanics of IPv4 packet fragmentation in Wireshark, and understand network layer forwarding behavior.

---

4. Topology Diagram



Details:

- 2x Hosts (`<initials>-H0` and `<initials>-H1`) communicating over a routed CML topology via Router (`<initials>-R0`).
- Both IPv4 and IPv6 routing are enabled across the network interfaces.

---

5. Equipment & Environment

- **Software:** Cisco Modeling Labs (CML), Terminal Emulator (e.g., PuTTY, SecureCRT, native terminal), Wireshark
- **Hardware / Virtual Nodes:** 2x Linux Nodes (configured as hosts), 1x IOSv Router

---

6. Configuration / Methodology

IP Addressing Table

Device	CML Node Name	Interface	IPv4 Address	IPv6 Address	Default Gateway
Client	`<initials>-H0`	eth0	10.0.1.10/24	2001:db8:1::10/64	10.0.1.1 / 2001:db8:1::1
Server	`<initials>-H1`	eth0	10.0.2.10/24	2001:db8:2::10/64	10.0.2.1 / 2001:db8:2::1
Router	`<initials>-R0`	G0/0	10.0.1.1/24	2001:db8:1::1/64	N/A
Router	`<initials>-R0`	G0/1	10.0.2.1/24	2001:db8:2::1/64	N/A

Step-by-Step Configuration Guide

1. Router Setup (`<initials>-R0`)

Open the **Console** tab of your Router node and configure it to enable dual-stack routing:

1. **Elevate to Privileged Mode**:

```
Router> enable
Router# configure terminal
```

2. **Set the Hostname**: Change the default hostname using your capitalized first and last initials in place of `KH` (e.g. `JD-R0` if your name is John Doe):

```
Router(config)# hostname KH-R0
```

3. **Enable IPv6 Routing**:

```
KH-R0(config)# ipv6 unicast-routing
```

4. **Configure GigabitEthernet0/0 (facing Host H0)**:

```
KH-R0(config)# interface GigabitEthernet0/0
KH-R0(config-if)# ip address 10.0.1.1 255.255.255.0
KH-R0(config-if)# ipv6 address 2001:db8:1::1/64
KH-R0(config-if)# no shutdown
KH-R0(config-if)# exit
```

5. **Configure GigabitEthernet0/1 (facing Host H1)**:

```
KH-R0(config)# interface GigabitEthernet0/1
KH-R0(config-if)# ip address 10.0.2.1 255.255.255.0
KH-R0(config-if)# ipv6 address 2001:db8:2::1/64
KH-R0(config-if)# no shutdown
KH-R0(config-if)# exit
```

6. **Save Configurations**:

```
KH-R0(config)# end
KH-R0# write memory
```

7. **Verify Interface Status**:
   - `show ip interface brief` (Check that both interfaces show status/protocol as **up/up**).
   - `show ipv6 interface brief` (Verify that IPv6 addresses are correctly configured and active).

> ***IMPORTANT - Deliverable Screenshot****: Take a screenshot of the CLI console showing the outputs of both the `show ip interface brief` and `show ipv6 interface brief` commands.*

---

2. Host H0 Configuration (`<initials>-H0`)

Click on your H0 client node, open the **Edit Config** tab, and enter the following shell configuration:

```
# Enable IPv4 static routing
ip addr add 10.0.1.10/24 dev eth0
ip route add default via 10.0.1.1

# Enable IPv6 static routing
ip -6 addr add 2001:db8:1::10/64 dev eth0
ip -6 route add default via 2001:db8:1::1
```

3. Host H1 Configuration (`<initials>-H1`)

Click on your H1 server node, open the **Edit Config** tab, and enter the following shell configuration:

```
# Enable IPv4 static routing
ip addr add 10.0.2.10/24 dev eth0
ip route add default via 10.0.2.1

# Enable IPv6 static routing
ip -6 addr add 2001:db8:2::10/64 dev eth0
ip -6 route add default via 2001:db8:2::1
```

Step-by-Step Traffic Generation & Execution:

1. **Start Packet Capture**: Right-click the link connecting **Host H0 and Router R0**, select **Packet Capture**, and click **Start**.
2. **Perform Standard IPv4 Ping**: Open Host H0's console and ping Host H1 over IPv4:

```
ping -c 4 10.0.2.10
```

3. **Perform Fragmented IPv4 Ping**: Send a ping packet with a large payload size (3000 bytes) to force IP fragmentation (since standard Ethernet MTU is 1500 bytes):

```
ping -c 1 -s 3000 10.0.2.10
```

4. **Perform Standard IPv6 Ping**: Ping Host H1 over IPv6:

```
ping6 -c 4 2001:db8:2::10
```

5. **Download Capture**: Stop the CML packet capture and download the `.pcap` capture file to your local computer.

---

7. Wireshark Analysis and Questions

Open your downloaded packet capture in Wireshark and answer the following questions:

Part A: Basic IPv4 Header Analysis

1. Select the first **IPv4 ICMP Echo Request** packet.
   - What is the value of the upper layer protocol field (e.g. `Protocol` field) in the IPv4 header?
   - How many bytes are in the IPv4 header?
   - How many bytes are in the payload of the IP datagram? Explain how you determined the number of payload bytes (Total Length minus Header Length).
   - Has this IP datagram been fragmented? Explain how you determined whether or not it was fragmented.
2. Inspect the sequence of IPv4 Echo Request packets sent from Host H0 to H1.
   - Which fields in the IP header *always* change from one datagram to the next in this sequence? Why?
   - Which fields in this sequence of IP datagrams stay *constant*? Why?
   - Describe the pattern you see in the values of the `Identification` field.
3. Inspect the **Time to Live (TTL)** value in the IPv4 header:
   - What is the TTL value of the packet as it leaves Host H0?
   - If you were to capture packets on the link between Router R0 and Host H1, what TTL value would you expect to see? Explain how the router modifies this value.
4. Now select one of the **IPv4 ICMP Echo Reply** packets returned from Host H1:
   - What is the upper layer protocol specified?
   - Are the values of the `Identification` fields across these reply packets similar in behavior (incrementing) compared to your observations of the requests?
   - Are the values of the TTL fields similar/identical across all of these response packets?

Part B: IPv4 Packet Fragmentation

5. Locate the packets generated by your `ping -s 3000` command.
   - Has the 3000-byte UDP/ICMP segment been fragmented across more than one IP datagram? How many IP fragments are generated for this single packet?
6. Inspect the IPv4 header of the **first** fragment:
   - What information in the IP header indicates that this datagram has been fragmented?
   - What information indicates whether this is the first fragment versus a latter fragment?
   - What is its `Total Length` (header plus payload) in bytes?
   - What is the state of the `More Fragments (MF)` flag, and what is the value of the `Fragment Offset` field?
7. Inspect the **second** fragment of the fragmented segment:
   - What information in the IP header indicates that this is *not* the first datagram fragment?
   - What fields change in the IP header between the first and second fragment?
8. Inspect the **third and final** fragment of the fragmented segment:
   - What information in the IP header indicates that this is the *last* fragment of that segment?

- How does the receiver know that all these fragments belong to the same original packet? (Hint: Compare their `Identification` fields).

Part C: IPv6 Header Analysis

9. Select the first **IPv6 Echo Request** packet.
    - What is the value of the `Next Header` field? How does this field function compared to the IPv4 `Protocol` field?
    - What is the size (in bytes) of the IPv6 header? How does it compare to the standard IPv4 header size?
10. Inspect the **Flow Label** field in this IPv6 datagram. What is its value?
11. Inspect the **Hop Limit** field of the IPv6 header. How does its functionality relate to the IPv4 TTL field?
12. Identify the source and destination IPv6 addresses:
    - Are these Global Unicast Addresses (GUA) or Link-Local Addresses (LLA)? How can you tell?

---

8. Troubleshooting

Issue Encountered: [Describe what went wrong, if anything] **Discovery Method:** [How did you find the issue?] **Resolution:** [How did you fix it?]

(If no issues were encountered, explain potential pitfalls for this configuration).

---

9. Conclusion & Analysis

Summarize your findings. Contrast the structure of IPv4 and IPv6 headers, detailing the advantages of IPv6's fixed header size, and explain how IP fragmentation allows data to span across link boundaries with varying MTUs.

---

10. What to Hand In

- The Lab YAML configuration file with your name, date, and name of the lab at the top in comments.
- A single PDF report containing:
    - Your written answers to the 12 Wireshark analysis questions in Section 7.
    - A screenshot of your active CML topology showing the running hosts (`<initials>-H0` , `<initials>-H1`) and router (`<initials>-R0`).
    - The required screenshot of the router CLI console displaying the interface brief lists as specified in Step 7.
    - An annotated screenshot of your Wireshark packet list showing the generated IPv4 fragments.
    - An annotated screenshot of the IPv6 Echo Request packet details pane in Wireshark, highlighting the Next Hop, Flow Label, and Hop Limit fields.